The Adversary's Companion

What the Open Items Buy You

A companion to *SIGIL — The Variational Chain*, whitepaper v1.3

v0 · 2026-07-24

`bitknight.dipper688@passmail.net`
SIGIL / Flux Foundation · `sigilgraph.quillon.xyz`

"An unmarked trust assumption is faith. A marked one is a debt."

— and this paper is the invoice

Scope & disclosure. This companion is a threat model of the SIGIL graph as it actually stands, written from the attacker's chair. It catalogues **8** open or partially-guarded attack surfaces, each rooted in a specific item from the project's own *hardening backlog* (2026-07-18, re-verified against live code) and security audit, with the exact `file:line` of the unguarded primitive quoted so each tree is re-checkable. It is pseudonymous and secret-free: it records attack *classes* and the cited open items, never live exploit payloads or secrets. Every attack carries a severity and a mitigation status; nothing open is argued away. A threat model is a disclosure document — publishing the surface, graded and dated, is how a system becomes accountable to close it. **The umbrella caveat governs everything below: `sigil-g0` is a live *experimental* network whose block-apply path is Phase-0 by design and does not yet enforce consensus cryptography. Do not route significant value through SIGIL until the items in §6 close.**

1	3	4
CATASTROPHIC — consensus or full-supply at stake	MAJOR — history substitution, issuance, or sync poisoning	MODERATE — replay, reward inflation, or light-client deception

Fifth companion to the SIGIL whitepaper. Cross-referenced with the philosophy companion (v0.3), the Failure Atlas (v0), and the Builder's Chronicle (v0). Sources and file:line anchors in §7.

1 The honest frame

A chain's whitepaper argues that it is safe. Its threat model argues the opposite, on purpose, and the distance between the two is the real measure of the thing. This document is the opposite argument. It is written the way an attacker reads a system: not "what does it promise," but "where does the promise not yet reach, and what does that gap buy me."

Three properties make this exercise honest rather than theatrical. First, every attack below is rooted in a *real* item — a line of code that is verifiably unguarded, or a guard that is written but not yet enforcing — with the `file:line` quoted from the current tree. There are no invented cryptographic breaks and no speculative attacks with no anchor in the record. Second, each attack is graded twice: by severity (what it costs if it lands) and by mitigation status (GUARDED, PARTIALLY GUARDED, OPEN), so a reader can tell a shipped defense from a dormant one at a glance. Third, the whole model sits under one load-bearing caveat, stated plainly so nothing below can be read out of context.

> IN PLAIN WORDS — **Why a chain would publish its own attack list**
>
> A silent weakness is still a weakness — it just has no deadline attached. Writing it down, with the exact code that is unguarded and how bad it would be, turns a vague risk into a debt the project owes and can be held to. The whitepaper already opens with a security disclosure telling people not to put real money in yet. This paper is that disclosure taken all the way down: the full itemized attack surface, so closing it is a public obligation, not a private hope.

The umbrella caveat

`sigil-g0` is a live *experimental* network. Its block-apply path is Phase-0 by design: the code comment at `sigil-node/chain.rs:84` reads "Phase 0 omits crypto verification," and `precheck()` validates byte-*lengths* only, not signatures, proof-of-work, or the verifiable-delay proof (`sigil-header:270-305`, `chain.rs:86`, `sigil-chronos lib.rs:333`). The audit states the consequence in one sentence: *"`sigil-g0` balances are not trust-bearing today."* The whitepaper's own security box says it out loud — *"block-apply does not yet enforce the producer's signature… peer-sync ingress is under-authenticated… Do not route significant value through SIGIL until they close."*

Every attack tree in §3 is a *specific* instance of this umbrella. That is deliberate: naming the eight concrete gaps is what turns "not trust-bearing yet" from a disclaimer into a checklist.

Where the boundary already holds

The honest frame cuts both ways: it must be as exact about what *is* guarded as about what is not. The RPC edge — the money-touching HTTP surface an anonymous caller can reach on the public daemon — *is* authenticated and enforcing. A per-request ed25519 wallet-signature over a domain-separated canonical message, plus a per-wallet nonce replay store, gate every spend route (`authorize()` at `sigil-rpc/src/bin/sigil-rpcd.rs:682`); the 2026-06-10 audit's headline finding — an anonymous `curl` minting to the 21M cap through a difficulty-zero `/mine` — was fixed and deployed the same day. The residual risk is therefore *not* at the edge. It is one layer down, in the consensus cryptography and the peer-sync ingress, which no external HTTP client touches but every gossiping peer does. This paper is about that layer.

How claims are graded

SIGIL papers grade every empirical statement on two axes: an evidence grade (● MEASURED measured on the live path, ⊙ SIMULATED shown in seeded simulation, or claimed-by-derivation) and a deployment grade (● LIVE live, ◇ DORMANT dormant/coded-but-not-active, or absent). A threat model inherits the discipline exactly. An attack reproduced in a controlled experiment is ● MEASURED; an attack that is the *claimed consequence* of a cited, real, unguarded item — but has not been run end-to-end — is stated as such and never dressed up as demonstrated. A mitigation that exists as tested code behind an activation gate is ◇ DORMANT PARTIALLY GUARDED, never ● LIVE GUARDED. The cardinal sin here is the category error, not the overstatement: "an attacker *could*" and "an attacker *did*" are different sentences, and this paper refuses to blur them.

2 How to read an attack tree

Each entry in §3 is one attack, in a fixed shape. The header carries the backlog ID, a one-line attacker goal, the `file:line` of the root primitive, a severity marker, and a mitigation-status marker. The body rows are always the same five:

GOAL	what money or consensus outcome the attacker achieves
PRECONDITIONS	the position, keys, or timing the attack requires
STEPS	how the exploit actually runs, concretely
BUYS	the blast radius — one wallet, the reward stream, a fork, the whole prefix
DEFENSE	the guard's true status: what is built, what is dormant, what is missing, and the height-gate strategy for a safe cutover

Severity: CATASTROPHIC (consensus or full-supply), MAJOR (history, issuance, or sync), MODERATE (replay, reward inflation, light-client deception), MINOR. Mitigation: GUARDED (shipped and enforcing), PARTIALLY GUARDED (primitive built, not wired or not enforcing), OPEN (genuinely unaddressed). Deployment/state: ● LIVE the defense is live, ◇ DORMANT the defense is dormant, ○ OPEN open, and ▲ EXPLOIT LIVE the *exploit* itself is still live on the current path. The trees are ordered by severity, worst first.

> IN PLAIN WORDS — **The pattern to watch for**
>
> Read a few entries and one shape repeats: the *fix usually already exists in the codebase* — written, unit-tested — but is switched off, waiting behind an activation height, or has no live caller. That is not an accident of this paper; it is the true shape of the surface, and §5 names it. "Built but not wired" is a very different risk from "nobody has thought about it," and a threat model earns its keep by telling them apart.

3 The attack trees

3.1 Consensus capture

H1 · Forge blocks the whole network applies `sigil-header:270-305` CATASTROPHIC ◇ DORMANT PARTIALLY GUARDED

GOAL	Inject arbitrary blocks that every follower applies — mint coinbase to attacker wallets, rewrite balances, or fork the chain. Full consensus capture.
PRECONDITIONS	A libp2p connection to a follower or producer, and the ability to gossip a block. *No key* — the producer signature field is not verified, so it need not be valid.
STEPS	(1) Craft a block with any `producer_sig` bytes — all-zero matches what the honest producer already writes. (2) Pass `precheck()`, which validates byte-lengths only. (3) Gossip it; followers apply it because the apply path runs no producer-signature verification. (4) Embed a coinbase or `SetBalance` mutation crediting attacker wallets.
BUYS	Everything — a single wallet up to the entire 21M cap, arbitrary forks, follower desync. This is the highest-severity item in the backlog.
DEFENSE	PARTIALLY GUARDED, ◇ DORMANT. The verifier exists — `sigil-header::verify_producer_sig` / `verify_at_height`, 7 tests green — but it is height-gated at `H1_PRODUCER_SIG_ACTIVATION_HEIGHT = u64::MAX`, so it never fires. Activation is blocked on three real pieces of work, all wiring not building: the producer must actually sign (today it writes a zeroed sig); the hot path must move to `Ed25519Hot` or gain a validator registry for the post-quantum schemes; and an Alpha-Docker soak must pass. The activation height is the coordinated cutover: historical zero-sig blocks keep validating under the old rule, blocks at or above the gate require a real signature. See `docs/H1_PRODUCER_SIG_GO_NO_GO.md`.

IN PLAIN WORDS — **Why this one is the whole game**

On a finished chain, a block is only accepted if the right producer signed it. SIGIL has written that check and tested it — but left it switched off, because the producer isn't signing yet. Until both halves are on, a block with a blank signature is accepted, which means someone who can talk to a node can hand it a block that says whatever they want. This is exactly why the umbrella caveat says not to put real money in yet: it is not a subtle flaw, it is the front door, and the paper lists it first for that reason.

3.2 History substitution & issuance

M2 · Serve a malicious "verified" snapshot `sigil-node/main.rs:1037` MAJOR ◇ DORMANT PARTIALLY GUARDED

GOAL — Feed a fast-forwarding node an attacker-chosen historical prefix that it adopts as chain history without ever cryptographically verifying it.

PRECONDITIONS — A node performing snapshot-pull fast-forward, and a peer position to serve the snapshot. Latent today: snapshot-pull is gated off on the fleet default until a signed anchor is published — but the M2 crypto gap is exactly what blocks safe activation.

STEPS — (1) Serve a snapshot whose skeleton records hash consistently — the attacker controls the `archive_root` computation over its own records. (2) With `anchor_sig` empty, nothing binds the snapshot to a trusted signer; with `fold_blob` empty, no fold checkpoint authenticates the prefix's chain validity. (3) The fast-forwarding node adopts the attacker prefix.

BUYS — Wholesale history substitution for a fast-forwarding node — the largest desync blast radius short of H1, because it rewrites the entire prefix, not one block.

DEFENSE — PARTIALLY GUARDED. The M1 structural layer is real: `main.rs:1030` serves a genuine `archive_root` (BLAKE3 over the ordered `SkeletonRecord`s, matching the client `SnapshotVerifier`). The M2 authentication is absent — `anchor_sig` and `fold_blob` are still empty (`:1037`), so fast-forward authenticates by hash only. Two pieces close it: (a) publish a real signed anchor — the operator pins the trust-root pubkey `SIGIL_ANCHOR_PK_HEX` plus a DNS/URL; (b) fill the trailer with a SQIsign signature and a real `flux_fold` checkpoint, and verify the fold on fast-forward. Because the feature is off until the anchor exists, this is not exploitable live yet — but it cannot be safely turned on until both land. Test gate: a wrong-signature snapshot rejected; a tampered prefix caught by fold-verify.

C10 · Instant-mine via a forgeable VDF modulus `flux-vdf/src/lib.rs:179` MAJOR ▲ EXPLOIT LIVE PARTIALLY GUARDED

GOAL — Collapse the time-lane. Produce "delayed" verifiable-delay outputs instantly — capture issuance, out-run honest miners, and break the timing assumption the tip-bound challenge seed rests on.

PRECONDITIONS — Knowledge of the `bench_2048` modulus's factorization — it is a benchmark modulus, so its factors are public or derivable — and a miner or block-producer role.

STEPS — (1) Because the group order is known, compute the Wesolowski VDF output directly instead of by sequential squaring. (2) Submit zero-real-delay "elapsed-time" proofs to `/mining/submit` or inside blocks. (3) Win the issuance race; the time-lane offers no independent rate limit.

BUYS — Issuance capture — the mining reward stream and any timing-gated consensus. Not a full-supply mint on its own, but it composes with H1.

DEFENSE — PARTIALLY GUARDED, and the exploit is still ▲ EXPLOIT LIVE. The trustless replacement is built: `flux-vdf/src/lib.rs:195` adds `rsa2048()` — the RSA-2048 Factoring-Challenge semiprime, unknown factorization, nothing-up-my-sleeve — plus `production()` and `assert_secure()` (`:226`, which rejects small/even/prime/smooth N and fails closed). But every live consensus and mining site still calls the forgeable `bench_2048()`: `sigil-rpcd.rs:291` (follower apply), `:1215` (`/mining/submit`), and every miner (`flux-miner/src/engine.rs:105,294`, `client.rs:211,231`, the miner bins). The fix is to wire `production()` into every verify site, height-gated: pre-activation blocks were mined against `bench_2048`, so a clean cutover needs a height boundary plus a coordinated miner upgrade. Small change, high blast radius — Alpha Docker first. Test gate: a `bench_2048`-shortcut proof rejected under the production modulus; an honest proof verifies.

H2 · Feed sync from an unauthenticated peer `sigil-node/src/sync_auth.rs` MAJOR
◇ DORMANT PARTIALLY GUARDED

GOAL Pose as a legitimate validator peer, serve a fabricated or forked block history over the backfill path to a syncing node, and desync or poison its state.

PRECONDITIONS A libp2p connection to a node that is backfilling or bootstrapping. While enforcement is off (the default), no valid handshake is needed.

STEPS (1) Connect to a syncing follower. (2) Answer its `BackfillReq` with attacker-chosen blocks; because the handshake gate is log-only by default, the unauthenticated serve is accepted and merely telemetry-flagged. (3) Combined with H1 or H3, deliver a fork the follower applies.

BUYS Follower desync or fork adoption on the sync path. The target is a bootstrapping or lagging node, not the steady-state producer.

DEFENSE PARTIALLY GUARDED. The crypto is real and on-branch: `verify_handshake` is ed25519 over the canonical transcript, fail-closed (a Blake3 stub yields `StubRejected`, an unwired PQ algorithm `UnsupportedAlgorithm`). The node mints a 12-hour `ValidatorPeer` handshake at boot (key `<db_path>/handshake_ed25519.key`, sync-only, not a wallet key), attaches it to every outgoing `BackfillReq`, and the server verifies and caches sessions with *channel binding* — the `session_pubkey` must equal the arriving libp2p peer id, so a cross-peer replay fails inside the validity window. 11/11 crate and 42/42 node tests. It is ◇ DORMANT because the default is log-only (authed/anon/refused telemetry) to let the fleet migrate; `SIGIL_HANDSHAKE_REQUIRE=1` refuses before any block transfer. Remaining: the `sigil-top` client attach, then the enforcement flip once telemetry shows followers authenticate.

3.3 Reward inflation, replay & light-client deception

H3 · Claim full reward for an easy share `sigil-rpcd.rs:150` MODERATE ◇ DORMANT PARTIALLY GUARDED

GOAL Serve a block with a trivially-low difficulty, have the follower verify the cheap proof-of-work and credit full reward — mint reward for near-zero work.

PRECONDITIONS A producer/peer role able to gossip a block with a chosen `bits` field; enforcement off (the default).

STEPS (1) Mine a block against an artificially low difficulty. (2) Declare the low `bits` in the block; the follower verifies the share against the *declared* difficulty, not an independently reconstructed target. (3) Collect full reward for cheap work.

BUYS Reward inflation and mining-economics distortion; with H1, also a cheap route to producing accepted blocks. The self-mining `/mining/submit` path was always safe — its target is server-derived.

DEFENSE PARTIALLY GUARDED. `apply_block` now runs `check_difficulty_floor` before verifying the share, enforcing the difficulty *envelope*: an absolute floor (`bits` ≥ `SIGIL_MIN_BLAKE4_BITS`, default 10 — which kills the named `bits=4` attack) and a per-block clamp (`bits` may not drop more than `RETARGET_MAX_STEP`, 2, below the last-accepted value, valid because the producer retargets only once per `RETARGET_WINDOW` of 16 blocks and by at most 2). Both bounds provably hold for honest blocks, so enforcement cannot fork off the honest chain. Pure checker, 5/5 unit tests, telemetry `H3_WOULD_REJECT`; enforce with `SIGIL_DIFFICULTY_ENFORCE=1`. Honest residual: the envelope *bounds* the attack but does not reconstruct the *exact* required target — that needs verified in-block timestamps and a replayable retarget rule, a consensus change that depends on the H7 timestamp guard. The cheap-share attack is closed; a subtler in-envelope manipulation is not yet provably excluded.

H5 · Replay a signed transaction across blocks `sigil-tx:18-20` MODERATE ◇ DORMANT PARTIALLY GUARDED

GOAL Replay a victim's already-valid signed transaction in a later block, re-executing the transfer to drain the sender or double-credit a recipient.

PRECONDITIONS Observe one valid signed transaction on the wire or in the mempool — no key needed, the attacker replays the victim's own signature — and a path to re-inject it into a later block.

STEPS (1) Capture a broadcast signed transaction. (2) Re-broadcast it after it is mined; the in-RAM mempool dedup does not span blocks or restarts. (3) The live apply path applies it again, because no persistent per-wallet nonce high-water is consulted.

BUYS Repeated unauthorized transfers, bounded by victim balance times replays — a single- or few-wallet blast radius, not the whole supply.

DEFENSE PARTIALLY GUARDED. The mechanism is sound and committed but has no live caller. `sigil-state::check_and_bump_nonce` (`lib.rs:336`) stores a per-wallet high-water as a reserved `NONCE_TOKEN` balance, so it rides the committed `wallet_state_root` and a forged `SetBalance` to `NONCE_TOKEN` is rejected (`:882`, `ReservedToken`); `sigil-tx::apply_signed_batch` (`:1360`) folds the nonce into the signed digest and rejects `nonce <= stored` (`NonceReplay`). Both are tested. The gap: neither has any caller in `sigil-node` or `sigil-rpcd`, so the live money path never runs them, and the legacy `SignedTx.nonce` field is still neither signed nor checked. The fix is wiring — route the live apply path through `apply_signed_batch`, or call `check_and_bump_nonce` in the RPC apply. Test gate: a replayed signed batch rejected at the live apply path.

H4 · Fabricate a wire tip-proof

`sigil-node/src/main.rs:1686` MODERATE ○ OPEN OPEN

GOAL Feed a light client or peer a forged tip-proof asserting a false chain tip or balance state; the client trusts it and answers balance queries against attacker-chosen state.

PRECONDITIONS The ability to gossip on `TOPIC_TIP_PROOFS` to a subscribing light client. No key required — the proof is a keyless BLAKE3 fingerprint.

STEPS (1) Fabricate a `Blake3Fingerprint` tip-proof over any tip or state you choose; nothing binds it to a key. (2) Publish it on the tip-proof topic. (3) A light client with no receiver-side verification accepts it as the chain tip and serves wrong balances.

BUYS Light-client and fresh-node deception — false balance queries, a false tip. It does not move consensus state on full nodes, but it undermines the whitepaper's headline fresh-node proof path.

DEFENSE OPEN — genuinely unaddressed, and this one is building, not merely wiring. The wire always builds `TipProof::new_blake3` (`main.rs:1686,1750`); the only `.verify()` (`:1788`) is the producer's own dry-run self-check, not receiver ingress; `TOPIC_TIP_PROOFS` appears only in the publish direction (`:1877`) with no verifying subscriber. `SqiSignBlob` exists as a flavor (`sigil-tip-proof:52`) but `new_sqisign` is never called for tip-proofs. The fix is to produce *and* verify `SqiSignBlob` tip-proofs on the wire, keeping BLAKE3 as integrity-only and never a trust root — which is exactly the contract the WS2 `/balance` verifier already gates on via `adversary_resistant()`. Test gate: a fabricated BLAKE3 tip-proof fails the adversary-resistant gate; a valid SQIsign tip-proof passes.

WS2 · Prove a balance against an uncommitted root

`sigil-state roots()` MODERATE

◇ DORMANT PARTIALLY GUARDED

GOAL	Serve a light client a `/balance?proof=1` response with a valid-looking inclusion proof against a balance state the chain tip does not actually commit to.
PRECONDITIONS	A light client relying on `/balance?proof=1`, and a node (or a machine-in-the-middle) serving the response.
STEPS	(1) The client requests a proof-carrying balance. (2) The server returns an SMT inclusion proof, which the client checks against an SMT root — but no *tip-attested* SMT root exists to bind that root to consensus. (3) A dishonest server presents a proof against an SMT root the real tip never committed to; the seam is the gap between "the proof verifies" and "the root is the canonical committed root."
BUYS	Light-client balance deception — the same victim class as H4. The proof machinery is real; the anchoring is missing. Not a full-node consensus break.
DEFENSE	PARTIALLY GUARDED. The read path is realized on-branch: `prove_balance()`, `/balance?proof=1`, and the `sigil-balance-verify` client gate on `TipProofFlavor::adversary_resistant()`. But the committed `wallet_state_root` is still the additive accumulator, not the SMT the proof references — so the tip-proof does not yet attest the SMT root. The fix is the height-gated Phase-3 swap the `roots()` documentation already anticipates: add the SMT root to the header/roots, maintaining the SMT incrementally in `set_balance`. It depends on H4 (SQIsign tip-proofs on the wire) to be end-to-end meaningful. Test gate: a light client trusts a `/balance?proof=1` response only after checking it against the tip-attested SMT root.

4 What we already attacked ourselves with

A threat model that only lists open items reads as if the system had never been tested. It has — adversarially, on purpose, and the results are the other half of the honest picture. Before the DagKnight braid-ordering layer was allowed near the live network, it ran a six-scenario adversarial simulation gate; on any state divergence a node self-halts with **exit code 78** (no auto-restart, operator alarm) before it serves a single byte. All six report divergence **= 0**:

SCENARIO	WHAT IT ATTACKS	RESULT
S1	10k blocks, 3 producers, dual-instance	**divergence = 0**
S5	30% drop / reorder / equivocation	**divergence = 0**
Orderings	16/16 permutations agree; incremental ≡ batch	**16/16**
Withheld-block attacker	finalized prefix byte-identical	**1378 blocks**
Tamper suite	mutated blocks must be rejected	**5/5 rejected**
S6	100k blocks, maximum reorg window	**258 ≤ 16384 bound**

Corroborating live evidence: a two-producer mesh ran **3385 emissions byte-identical** before the experimental network's producer flipped to the braid binary at height **17.52M** (2026-07-02). The honesty caveat travels with the result: *simulation convergence is encouraging evidence, not a safety or liveness proof* — the Byzantine-threshold theorem, the fold-soundness reduction, and the composition bound remain on the open-theory rung of the evidence ladder. What the gate demonstrates is narrower and real: across these adversaries, the ordering layer did not diverge, and where it might, it stops rather than lies.

Three attack classes the audit once listed are now closed outright, and are recorded here so the reader does not mistake a stale finding for a live one — the current tree wins over the older audit on every disagreement:

- **Bridge mint (C9)** — guarded. `sigil-bridge/src/lib.rs`: `process_deposit` (`:82`) binds amount and recipient to the proven transaction via `deposit_intent()` (not caller arguments), enforces a spent-set, gates to BTC-only, and takes a PoW floor; `process_withdrawal` (`:124`) requires the owner's ed25519 signature plus an idempotent id; `process_ln_deposit` (`:146`) binds to a signed BOLT11 invoice plus spent-set. **25 bridge tests** green. The audit's "unbounded / no replay / no sig" is stale.
- **Event-log root (H6)** — guarded. `sigil-state::hash_event_log` (`lib.rs:1089`) builds a positional balanced binary Merkle tree; `sigil-events` builds the identical tree and its inclusion roundtrip is tested (`:527`), with tamper-rejection at `:546`. The audit's "order-blind accumulator" premise is factually stale. (Minor residual: event ordering is positional-but-producer-chosen, not independently constrained.)

- **RPC theft routes** — guarded. The wallet-signature auth gate deployed enforcing on the production daemon 2026-06-10 closed the anonymous mint, the owner-less swap and liquidity spend, and the uncapped token deploy. The residual, disclosed in §6: the `/onboard` faucet and two demo routes.

5 The shape of the surface

Read the eight trees together and a single shape dominates. Of the eight, one is OPEN (H4), and *seven are* PARTIALLY GUARDED — the defensive primitive is written and tested, but dormant, unwired, or enforcing only in log-only mode. This is the same shape the Failure Atlas found across seventy-one incidents, where "built but not wired" was the single largest failure class. A threat model earns its keep by naming that distinction precisely, because the two risks demand different responses. An OPEN item needs design and implementation; a PARTIALLY GUARDED item needs a decision, a soak, and an activation height.

The recurring instrument is the *height gate*. Every consensus-touching fix here — producer signatures (H1), the trustless VDF modulus (C10), the committed SMT root (WS2) — lands behind an activation height rather than as an immediate rule change. This is not timidity; it is the only way to change a validation rule on a live chain without orphaning the history that was produced under the old rule. Pre-activation blocks keep validating by the old rule, at-or-after blocks by the new one, and the boundary is a scheduled, revocable, coordinated cutover. The cost is that a written defense sits visibly dormant for a while — which is precisely the state this paper is built to disclose rather than hide.

One residual is disclosed rather than closed, because closing it fights the cold-start problem head-on: the `/onboard` faucet still credits new wallets a fixed grant per call, which a caller with fresh addresses can farm. A new wallet cannot pre-authenticate — it has no funds and no history — so the honest fix is a finite, operator-funded faucet with a per-IP rate limit and a debit against a real reserve, not an unbounded mint. Until that ships, the faucet is a disclosed, bounded-value soft spot, not a silent one.

> IN PLAIN WORDS — **What "partially guarded" really costs**
>
> Seven of the eight weaknesses already have their fix written and tested — it is just turned off. That is good news and a warning at once. Good, because closing them is mostly a matter of decisions and careful rollout, not invention. A warning, because a defense that is switched off protects nobody, and it is easy to feel safe looking at code that never runs. This paper's job is to keep the gap between "the fix exists" and "the fix is on" visible, dated, and owed.

6 Still open — the surface without a guard yet

The disclosure, consolidated. Nothing here is argued away; each row is the honest current status, worst first.

ID	SEVERITY	ATTACK SURFACE	STATUS
H1	CATASTROPHIC	Block-apply does not enforce the producer signature (verifier dormant at `u64::MAX`)	◇ DORMANT PARTIALLY GUARDED
M2	MAJOR	Snapshot fast-forward has empty `anchor_sig` / `fold_blob`	◇ DORMANT PARTIALLY GUARDED
C10	MAJOR	Live VDF still runs over the forgeable `bench_2048` modulus	▲ EXPLOIT LIVE PARTIALLY GUARDED
H2	MAJOR	Verify-before-sync handshake enforcing in log-only mode	◇ DORMANT PARTIALLY GUARDED
H3	MODERATE	Follower bounds but does not re-derive the exact difficulty target	◇ DORMANT PARTIALLY GUARDED
H5	MODERATE	Nonce-replay primitive committed but has zero live callers	◇ DORMANT PARTIALLY GUARDED
H4	MODERATE	Keyless wire tip-proofs; no receiver-side verification	○ OPEN OPEN
WS2	MODERATE	SMT root not committed in the header, so `/balance` proofs are unanchored	◇ DORMANT PARTIALLY GUARDED
—	MINOR	`/onboard` faucet uncapped; two demo routes ungated	○ OPEN OPEN

The umbrella over the whole table stands: `sigil-g0` is experimental and its balances are not trust-bearing until the consensus-crypto rows (H1, C10) and the peer-sync rows (H2, H3, H4, M2) close. That sentence is the debt this paper exists to keep on the books.

7 Closing

The distance between a whitepaper and its threat model is where a project's honesty lives. A system that only publishes its promises asks to be trusted; a system that publishes, grades, and dates its attack surface asks to be *checked*, and gives the reader the file and line to do it with. Every attack in this companion is a specific instance of one disclosed sentence — that `sigil-g0` does not yet enforce its consensus cryptography — turned into an itemized, severity-ranked, mitigation-graded list. Seven of the eight surfaces already have their guard written and waiting behind an activation height; one is still open. None is hidden.

That is the whole argument. An attack surface you can see is a roadmap; an attack surface you cannot is a liability. This paper chose the first on purpose.

v0 — 2026-07-24. The verifier is written; the gate is at `u64::MAX`; the debt is on the ledger. A disclosed surface is a roadmap — a silent one is a liability. Publish the attacks, grade them, date them, and let the closing of them be owed in public.

Sources

- *SIGIL Hardening Backlog*, 2026-07-18 — the ground-truth doc, every verdict re-verified against the live tree; the spine of this threat model. `docs/SIGIL_HARDENING_BACKLOG.md`.
- *SIGIL Security Audit*, 2026-06-10 — the earlier six-agent audit; several findings since fixed (the backlog wins on any disagreement). `SECURITY_AUDIT_2026-06-10.md`.
- *SIGIL — The Variational Chain*, whitepaper v1.3 (2026-07-22) — the design this threat model probes; its security-disclosure box and honesty section are the canonical open-item list. https://quillon.xyz/downloads/SIGIL_WHITEPAPER_v1.pdf.
- *What the Chain Knows — The Philosophy of the SIGIL Graph*, v0.3 (2026-07-22) — the epistemology of honesty this paper's grading inherits. https://quillon.xyz/downloads/SIGIL_PHILOSOPHY_v0.pdf.
- *The SIGIL Failure Atlas*, v0 (2026-07-23) — the sibling companion: 71 real failures in ten classes, where "built but not wired" is the same dominant shape found here. https://quillon.xyz/downloads/SIGIL_FAILURE_ATLAS_v0.pdf.
- *The Builder's Chronicle*, v0 (2026-07-23) — the fourth companion: development settled on-chain as a prototype labor institution. https://quillon.xyz/downloads/SIGIL_BUILDERS_CHRONICLE_v0.pdf.
- *The Measurement Book*, v0 (2026-07-24) — the sixth companion: the measured record, whose pretend inventory is the same open-item list this paper attacks. https://quillon.xyz/downloads/SIGIL_MEASUREMENT_BOOK_v0.pdf.